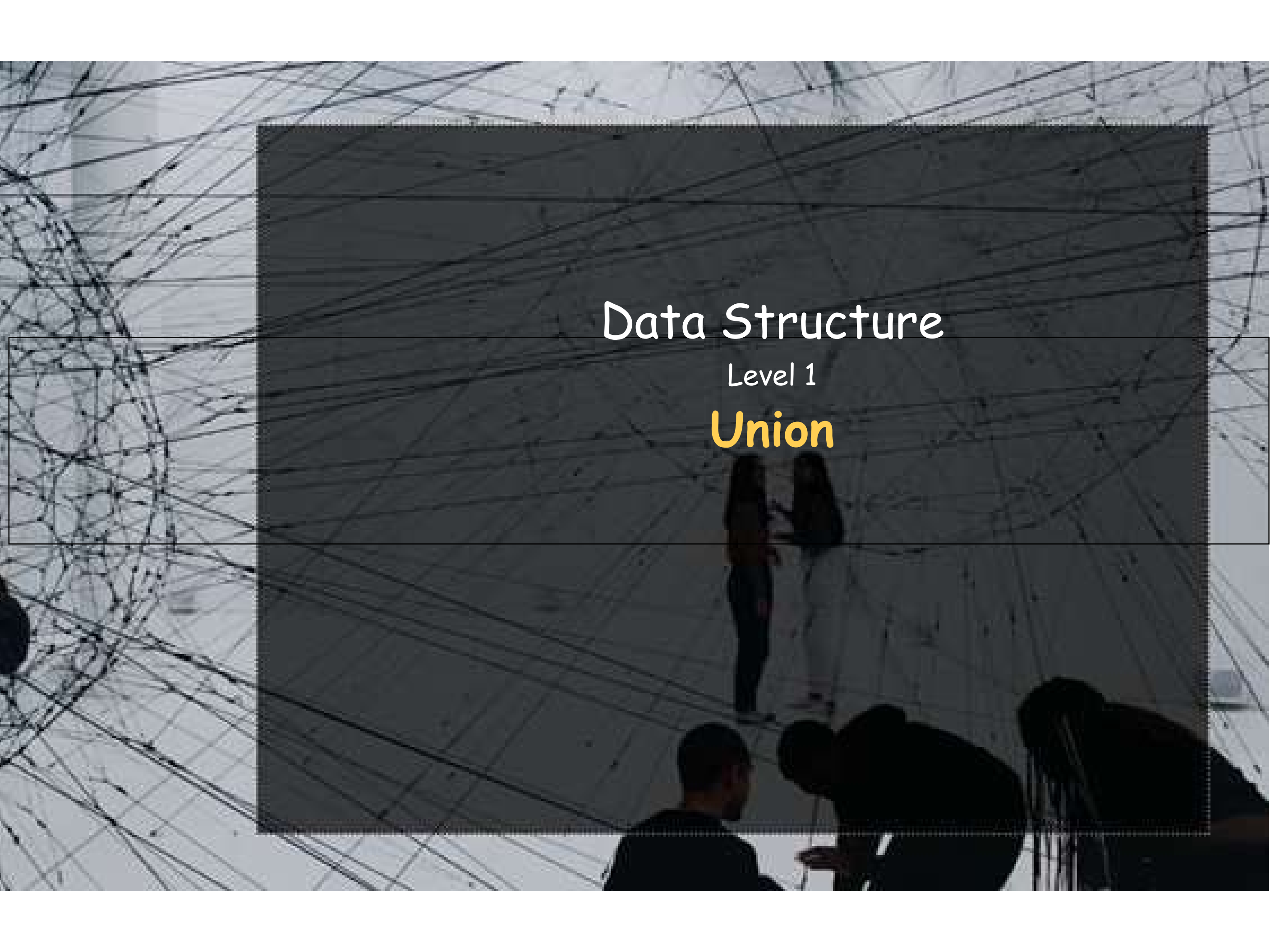


Data Structure

Level 1

Union



Data Structure

Level 1

Union

Definition

A union is a user-defined data type in C++ that allows different members to share the same memory location.

⚡ Unlike a `struct`, where each member has its own memory space, all members of a union use the same memory space.

💡 Example:

```
union MyUnion {  
    int intValue;  
    float floatValue;  
    char charValue;  
};
```

➡ This union can store an `int`, `float`, or `char`, but only one member should be considered active at a time.

Explanation

◆ All members of a union share the same memory.

▣ This means:

▨ Only one member is normally used at a time

▨ Assigning a value to one member changes the shared memory

▨ The size of the union is based on its largest member

▨ Unions can save memory when different values are needed at different times

💡 Example:

```
MyUnion myUnion;
```

```
myUnion.intValue = 42;
```

➡ The shared memory now contains the value of `intValue`.

If we then write:

```
myUnion.floatValue = 3.14f;
```

➡ The same memory is reused for `floatValue`.

✦ The previous `intValue` is no longer the active value.

Procedures (Union)

1 Define a Union

■ Create the union and its members:

```
union MyUnion {  
    int intValue;  
    float floatValue;  
    char charValue;  
};
```

➔ Result: One shared memory area can be used by all three members.

2 Create a Union Object

■ Create an object:

```
MyUnion myUnion;
```

3 Assign a Value

■ Assign a value to one member:

```
myUnion.intValue = 42;
```

➔ Result: `intValue` becomes the active member.

4 Access the Value

■ Read the same member:

```
cout << myUnion.intValue;
```

➔ Result:

42

☰ Procedures (Union)

5 ☐ Assign Another Member

▣ Assign a different member:

```
myUnion.floatValue = 3.14f;
```

➡ Result: The same memory is now being used for `floatValue`.

☰ Procedures (Union)

6 ☐ Use Another Member

▣ You can then access:

```
cout << myUnion.floatValue;
```

➡ Result:

3.14

💡 Example

```
union MyUnion {  
    int intValue;  
    float floatValue;  
    char charValue;  
};  
  
int main() {  
    MyUnion myUnion;  
    myUnion.intValue = 42;  
    cout << myUnion.intValue << endl;  
    myUnion.floatValue = 3.14f;  
    cout << myUnion.floatValue << endl;  
    myUnion.charValue = 'A';  
    cout << myUnion.charValue << endl;  
}
```

Note

◆ The size of a union is determined by its largest member, plus any alignment requirements.

💡 For example:

```
union MyUnion {  
    int intValue;  
    float floatValue;  
    char charValue;  
};
```

▣ `int` → typically 4 bytes

▣ `float` → typically 4 bytes

▣ `char` → 1 byte

➡ The union will typically require 4 bytes on common systems.

Comparison

Union

- ✓ All members share the same memory
- ✓ Only one member is normally active at a time
- ✓ Size is based mainly on the largest member
- ✓ Can save memory

Struct

- ✓ Each member has separate memory
- ✓ All members can hold values simultaneously
- ✓ Size includes storage for all members
- ✓ Easier and safer for storing multiple values

⊘ Warning

⚠ Be careful when reading a union member that is not the member you most recently stored.

💡 Example:

```
myUnion.intValue = 42;
```

```
cout << myUnion.floatValue;
```

✦✦ Summary

- ✓ Union allows different members to share the same memory
- ✓ Only one member should normally be considered active at a time
- ✓ Assigning one member reuses the shared memory
- ✓ The size is based on the largest member and alignment
- ✓ Unions can save memory
- ✓ Be careful when accessing a member that is not the active member

Thank you 😊